Day 11 – Recursion Advanced

1️⃣ Sum of N Numbers (Recursion)

Problem:

1 se N tak ka sum find karo

Java Code

```
public class SumN {
    public static int sum(int n){

        if(n == 0) return 0; // base case

        return n + sum(n-1);
    }

    public static void main(String[] args) {
        System.out.println(sum(5));
    }
}
```

🔥 Dry Run

sum(5)
= 5 + sum(4)
= 5 + 4 + sum(3)
= 5 + 4 + 3 + sum(2)
= 5 + 4 + 3 + 2 + sum(1)
= 5 + 4 + 3 + 2 + 1
= 15

2️⃣ Power of Number (a^b)

Problem:

a^b calculate karo

Java Code

```
public class Power {
    public static int power(int a, int b){

        if(b == 0) return 1;

        return a * power(a, b-1);
    }

    public static void main(String[] args) {
        System.out.println(power(2, 5));
    }
}
```

🚀 Optimized Power (Important)

```
public class PowerOptimized {
    public static int power(int a, int b){

        if(b == 0) return 1;

        int half = power(a, b/2);

        if(b % 2 == 0){
            return half * half;
        } else {
            return a * half * half;
        }
    }
}
```

Time Complexity: O(log n) ✅

3️⃣ Check Array Sorted or Not

```
public class CheckSorted {
    public static boolean isSorted(int[] arr, int i){

        if(i == arr.length - 1) return true;

        if(arr[i] > arr[i+1]) return false;

        return isSorted(arr, i+1);
    }

    public static void main(String[] args) {
        int[] arr = {1,2,3,4};
        System.out.println(isSorted(arr, 0));
    }
}
```

4️⃣ First Occurrence of Element

```
public class FirstOccurrence {
    public static int find(int[] arr, int key, int i){

        if(i == arr.length) return -1;

        if(arr[i] == key) return i;

        return find(arr, key, i+1);
    }
}
```

5 Last Occurrence of Element

```
public class LastOccurrence {
    public static int findLast(int[] arr, int key, int i){

        if(i == arr.length) return -1;

        int idx = findLast(arr, key, i+1);

        if(idx == -1 && arr[i] == key){
            return i;
        }

        return idx;
    }
}
```

---

6 Remove Duplicates from String

```
public class RemoveDuplicates {

    public static void removeDup(String str, int idx, String result, boolean[] map){

        if(idx == str.length()){
            System.out.println(result);
            return;
        }

        char curr = str.charAt(idx);

        if(map[curr - 'a'] == true){
            removeDup(str, idx+1, result, map);
        } else {
            map[curr - 'a'] = true;
            removeDup(str, idx+1, result + curr, map);
        }
    }

    public static void main(String[] args) {
        removeDup("aabbcc", 0, "", new boolean[26]);
    }
}
```

---

7 Important Pattern (VERY IMPORTANT)

👉 Recursion me 3 steps yaad rakho:

1. Base case
2. Work (processing)
3. Recursive call